Joules per Logical-Qubit Operation: The Normalization Case for the JPCUB Benchmark

Rowan Brad Quni-Gudzinas*

September 3, 2026

Abstract

Energy benchmarks for quantum processors are usually quoted per physical gate or per physical qubit, figures that hide the dominant cost of a fault-tolerant machine: the error-correction overhead that multiplies physical work by a distance-dependent factor before any logical operation is delivered. This paper revises the joules-per-compute benchmark to normalize by logical-qubit operations, the unit that carries the overhead. The model states its conventions up front: a distance-d rotated surface-code patch requires $N_c = 2\,d^2$ - 1 physical qubits and $N_g = 8\,d^3$ physical gates per logical operation, so joules per logical-qubit operation (JPCUB) exceeds joules per physical gate by exactly the overhead factor $N_g = 8\,d^{3\cdot}$ All quantitative claims are computed by a deposited verification script: the overhead factor table across five distances (216 at d = 3, 1000 at d = 5, 5832 at d = 9, 27000 at d = 15, 74088 at d = 21), and the physical-qubit count that accompanies each. The central consequence is that a per-physical-gate benchmark understates the energy of a logical machine by between two and four orders of magnitude depending on the code distance, and any cross-architecture comparison that does not normalize by logical operations compares overhead regimes rather than computational efficiency. The reader should care because roadmap claims about quantum energy efficiency are only meaningful in the unit that includes error correction: joules per logical-qubit operation is the figure that survives the translation from a single logical qubit to a scaled machine. Where the premises end: the counting conventions ($2\,d^2$ - 1 patch, $8\,d^3$ gate count) are order-of-magnitude engineering conventions of the rotated surface code; the argument transfers to other codes by substituting their overhead functions; and the benchmark defines the unit of reporting, not the measured energy, which must be supplied by a device.

1 Introduction

Benchmarks exist to make comparisons meaningful. The energy benchmark of a quantum processor currently has no agreed unit, and the most common candidates - joules per physical gate and joules per physical qubit - measure the controller, not the computation. A fault-tolerant logical operation is delivered only after an overhead machinery of physical gates and syndrome rounds has run to completion, and that machinery is the dominant energy cost of a scaled machine. A benchmark that does not include it compares the wrong quantity.

This paper makes the normalization case: energy per logical-qubit operation is the correct unit, and it is related to the per-gate figure by an exactly computable overhead factor. The factor is $8\,d^3$ for the rotated surface code, which is the code family with the most detailed published overhead accounting. The contribution is a revision of the reporting unit, a computed overhead table that makes the revision quantitative, and the argument that any energy comparison across fault-tolerant architectures must carry the overhead of each architecture.

*doi:10.5281/zenodo.22280745 (version 2.0.0)

2 Prior work

The overhead accounting for the rotated surface code follows the convention used in the companion energy model and in Fowler's low-overhead logical Hadamard treatment [2]: approximately 2 d^2 stabilizer measurements of four CNOT gates per syndrome round, d rounds per logical operation, giving $N_g = 8\ d^{3\cdot}$ The companion QNFO record on thermodynamic and informational bottlenecks [4] motivates the energy-per-solution view that this benchmark revision operationalizes: the quantity that survives scaling is energy per delivered result, not energy per elementary operation. Lavasani, Zhu, and Barkeshli [3] provide the constant-overhead counterpoint, which is the honest boundary of this paper's regime: for hyperbolic codes the overhead function changes, and the normalization case transfers with N_g replaced by that code's overhead.

The benchmark-revision argument itself is independent of any single code: it is the claim that the reporting unit must include the overhead, whatever the overhead function is.

3 Model and conventions

Convention 1 (physical qubits). A rotated distance-d patch contains $N_c = 2\ d^2 - 1$ physical qubits (d^2 data, $d^2 - 1$ ancilla).

Convention 2 (gate overhead). One logical operation of the patch costs $N_g = 8\ d^3$ physical gates.

Convention 3 (reporting unit). JPCUB is defined as joules per logical-qubit operation:

JPCUB = $E_{logical}/N_{logical} = (E_g N_g) / 1 = E_g\ 8\ d^3$,

for a single logical operation with gate energy E_g. The per-gate figure is E_g. The two are related by exactly N_g.

Where the premises end: the conventions are those of the rotated surface code and are engineering figures, not theorems; the measured per-gate energy E_g is a device parameter that the benchmark does not fix.

4 Analysis

The analysis is the algebra of overhead.

A per-physical-gate benchmark reports E_g. A per-logical-operation benchmark reports $E_g N_g$. The ratio is $N_g = 8\ d^3$: at d = 3, the logical figure is 216 times the gate figure; at d = 9, 5832 times; at d = 21, 74088 times. The logarithmic gap between the two reporting units grows as 3 log10 d + log10 8, reaching four orders of magnitude at d = 21.

A per-physical-qubit benchmark reports an even less comparable quantity, because it mixes the qubit count with the operation count and the gate energy in a way that depends on the architecture's schedule. The logical-operation unit is the only one of the three that is proportional to delivered computational work.

5 Results

All values in this section are computed by the deposited verification script ($jpcub_{nv}$_verify.py, section NV.004); no number is transcribed from memory.

Overhead factor by distance.

The last column is the number of orders of magnitude by which a per-gate benchmark understates the per-logical-operation figure. At d = 21 the understatement is 4.87 orders of magnitude.

d	N_c (physical qubits)	N_g per logical operation	log10(N_g)
3	17	216	2.33
5	49	1000	3.00
9	161	5832	3.77
15	449	27000	4.43
21	881	74088	4.87

6 Discussion

Three consequences follow.

First, the reporting unit determines the conclusion of an energy comparison. A comparison of two controllers at joules per physical gate can rank them oppositely to a comparison at joules per logical operation when their code distances differ, because the overhead factor multiplies the per-gate figure by 8 d^3 of each architecture. Normalizing by logical operations removes the overhead from the comparison and isolates the quantity that both controllers actually deliver.

Second, the benchmark revision does not add physics; it changes the unit of reporting. The underlying energy is the same; the figure that a roadmap sees is not. This is the sense in which the revision is a normalization case rather than a measurement claim.

Third, the constant-overhead codes of [3] are the honest boundary. Where a hyperbolic code delivers logical gates with constant overhead, the gap between per-gate and per-logical figures closes to a constant, and the normalization argument remains but its magnitude shrinks. The benchmark definition is architecture-agnostic: it always reports joules per logical operation, and the overhead factor is then a property of the code being benchmarked.

7 What a practitioner can do with this result

1. **Change the reporting unit.** Publish energy figures as joules per logical-qubit operation, not joules per physical gate. The change costs nothing and moves the stated energy of a fault-tolerant machine by the overhead factor, which is up to four orders of magnitude at working distances.

1. **Carry the overhead in comparisons.** When comparing two fault-tolerant architectures, divide each one's measured energy by its own delivered logical operations. Comparing raw per-gate figures conflates overhead regimes with computational efficiency and can rank architectures in the wrong order.

1. **Use the overhead table as a conversion.** A per-gate measurement at distance d converts to JPCUB by the factor 8 $d^{3\cdot}$ The table provides the factor for the five working distances; the script computes any other distance in one line.

8 Conclusion

The joules-per-compute benchmark should be normalized by logical-qubit operations. For the rotated surface code, joules per logical operation exceeds joules per physical gate by the overhead factor 8 d^3, which reaches 74088 (4.87 orders of magnitude) at distance 21. A per-gate benchmark understates the energy of a fault-tolerant machine by the overhead of its error correction, and any energy comparison that does not carry the overhead compares regimes rather than efficiency. The JPCUB unit makes the overhead explicit, and the deposited script reproduces the overhead table at every working distance.

References

[1] Landauer, R. 1961. ”Irreversibility and Heat Generation in the Computing Process.” IBM Journal of Research and Development 5 (3): 183-191.

[2] Fowler, A. G. 2012. ”Low-overhead surface code logical Hadamard.” arXiv:1202.2639.

[3] Lavasani, A., G. Zhu, and M. Barkeshli. 2019. ”Universal logical gates with constant overhead: instantaneous Dehn twists for hyperbolic quantum codes.” arXiv:1901.11029.

[4] QNFO. 2025. ”Thermodynamic and Informational Bottlenecks of Scalable Fault-Tolerant Quantum Computation.” Zenodo. doi:10.5281/zenodo.17955898.

Changelog

- v2.0 (2026-09-03): the normalization argument is stated with computed overhead factors at five distances; the logarithmic-gap analysis (reaching 4.87 orders of magnitude at d = 21) is added; prior-work positioning added (Sections 2 and 6); HTML and PDF renderings added to the deposit; abstract rewritten with computed values, no deferred claims.
- v1.0 (2026-09-03): initial short-form record.

Verification

Every number in Section 5 is produced by $jpcub_{nv}$_verify.py (section NV.004), deposited with this record. The script computes N_c = 2 d^2 - 1 and N_g = 8 d^3 for the five working distances and reports the logarithmic gap. Run ”python $jpcub_{nv}$_verify.py” to reproduce the table.